

AI/ML + n8n AI Interview Prep

26 likely questions with answers and explanations — SQL, ML Fundamentals, LLMs/GenAI, and n8n / Agentic AI

Section 1 — SQL

Q1.

What is the difference between INNER JOIN and LEFT JOIN?

Answer:

INNER JOIN returns only rows that have matching values in both tables. LEFT JOIN returns all rows from the left table, and matched rows from the right table; unmatched right-side columns appear as NULL.

Why / Explanation:

This is the most commonly asked SQL question. Interviewers want to check you understand that LEFT JOIN 'protects' the left table from row loss, which matters when you need to see records with no match (e.g. customers with zero orders).

Q2.

What is the difference between WHERE and HAVING?

Answer:

WHERE filters individual rows before any grouping happens. HAVING filters groups after GROUP BY has produced aggregated values like COUNT() or SUM().

Why / Explanation:

You can't use an aggregate function inside WHERE (e.g. WHERE COUNT() > 5 is invalid) because WHERE runs before aggregation. HAVING exists specifically to filter on the aggregated result.*

Q3.

What does GROUP BY do, and what's a simple example?

Answer:

GROUP BY collapses rows that share the same value in specified column(s) into a single summary row, usually paired with an aggregate function. Example: SELECT department, COUNT(*) FROM employees GROUP BY department; gives the employee count per department.

Why / Explanation:

Every column in the SELECT list that isn't inside an aggregate function must appear in the GROUP BY clause — a very common gotcha interviewers test for.

Q4.

What is a subquery, and where can it be used?

Answer:

A subquery is a query nested inside another query. It can appear in SELECT, FROM, WHERE, or HAVING clauses, and can return a single value, a single row, or a full table.

Why / Explanation:

Example: SELECT name FROM employees WHERE salary > (SELECT AVG(salary) FROM employees); The inner query computes the average first, then the outer query filters against it.

Q5.

What's the difference between a window function and GROUP BY?

Answer:

A window function (e.g. `AVG(salary) OVER (PARTITION BY department)`) computes an aggregate value per row while still returning every individual row. `GROUP BY` collapses multiple rows into one summary row per group.

Why / Explanation:

Window functions let you compare an individual row to its group context, e.g. 'show each employee's salary next to their department's average' — something plain `GROUP BY` cannot do in a single query.

Q6.**What are primary keys and foreign keys?****Answer:**

A primary key uniquely identifies each row in a table and cannot be NULL or duplicated. A foreign key is a column in one table that references the primary key of another table, enforcing a relationship between the two.

Why / Explanation:

Foreign keys are what make `JOINS` meaningful — they define how tables relate to each other, and interviewers often ask this alongside a `JOIN` question to test relational thinking.

Section 2 — Machine Learning Fundamentals

Q7.

What is the difference between supervised, unsupervised, and reinforcement learning?

Answer:

Supervised learning trains on labeled data to predict an outcome (classification/regression). Unsupervised learning finds patterns or structure in unlabeled data (clustering, dimensionality reduction). Reinforcement learning learns via trial and error, receiving rewards or penalties for actions taken in an environment.

Why / Explanation:

This is often the opening ML question. Keep the answer to one clear sentence per type — interviewers are checking for a crisp mental model, not a lecture.

Q8.

What is overfitting, and how do you prevent it?

Answer:

Overfitting is when a model learns the training data too well, including its noise, and performs poorly on new/unseen data. It's prevented with techniques like regularization (L1/L2), dropout, cross-validation, gathering more training data, or simplifying the model.

Why / Explanation:

Interviewers like to pair this with 'underfitting' (model too simple, fails to capture patterns even in training data) to see if you understand the bias-variance tradeoff.

Q9.

Explain precision, recall, and F1-score. When would you prioritize recall over precision?

Answer:

Precision = $TP / (TP + FP)$ — of all predicted positives, how many were correct. Recall = $TP / (TP + FN)$ — of all actual positives, how many were caught. F1-score is the harmonic mean of the two. You'd prioritize recall when missing a positive case is costly, like disease detection or fraud detection.

Why / Explanation:

This tests whether you understand that accuracy alone is misleading on imbalanced datasets (e.g. 99% accuracy is meaningless if only 1% of cases are actually positive).

Q10.

What is gradient descent?

Answer:

Gradient descent is an optimization algorithm that iteratively adjusts a model's parameters in the direction that reduces the loss function, using the gradient (slope) of the loss with respect to each parameter.

Why / Explanation:

A good follow-up is knowing that 'learning rate' controls the step size — too high and it overshoots the minimum, too low and training is very slow.

Q11.

What is the bias-variance tradeoff?

Answer:

Bias is error from overly simplistic assumptions (underfitting); variance is error from being too sensitive to training data fluctuations (overfitting). Reducing one typically increases the other, so the goal is to find a balance that minimizes total error.

Why / Explanation:

This is a favorite theory question because it connects directly to overfitting/underfitting and shows you understand model complexity tradeoffs, not just definitions.

Q12.**What is a confusion matrix?****Answer:**

A table that shows the counts of true positives, true negatives, false positives, and false negatives for a classification model, letting you calculate accuracy, precision, recall, and F1 from it directly.

Why / Explanation:

Be ready to sketch a 2x2 grid on a whiteboard or explain each cell verbally — it's one of the most practical, hands-on questions in ML interviews.

Q13.**What are activation functions, and why are they needed in neural networks?****Answer:**

Activation functions (ReLU, sigmoid, softmax, tanh) introduce non-linearity into a neural network. Without them, stacking multiple layers would be mathematically equivalent to a single linear layer, no matter how many layers you add.

Why / Explanation:

Know at least ReLU (used in hidden layers, fast and avoids vanishing gradients) and softmax (used in the output layer for multi-class classification, converts scores into probabilities).

Q14.**What is regularization, and what's the difference between L1 and L2?****Answer:**

Regularization adds a penalty term to the loss function to discourage overly complex models and reduce overfitting. L1 (Lasso) adds the absolute value of weights and can shrink some weights to exactly zero (feature selection). L2 (Ridge) adds the squared value of weights and shrinks them smoothly toward zero without eliminating them.

Why / Explanation:

If asked 'which would you use to also do feature selection', the answer is L1, since it can zero out irrelevant features entirely.

Section 3 — LLMs & Generative AI

Q15.

What is a Large Language Model (LLM), and how does it generate text?

Answer:

An LLM is a neural network (typically a transformer) trained on massive amounts of text to predict the next token in a sequence. Text generation happens by repeatedly predicting the most likely next token given all previous tokens, one token at a time.

Why / Explanation:

Interviewers want to hear 'next-token prediction' and 'transformer architecture' specifically — these are the two anchor phrases that signal real understanding.

Q16.

What is RAG (Retrieval-Augmented Generation)?

Answer:

RAG is a technique where relevant documents or data are retrieved from an external knowledge source (often a vector database) and injected into the LLM's prompt as context before it generates a response.

Why / Explanation:

RAG's main purpose is to reduce hallucination and let the model answer using up-to-date or private information it wasn't originally trained on — a very likely follow-up question.

Q17.

What are embeddings, and why are they used?

Answer:

Embeddings are numerical vector representations of text (or images) that capture semantic meaning, so that pieces of text with similar meaning end up close together in vector space. They're used for semantic search, clustering, and retrieval in RAG systems.

Why / Explanation:

A good example to mention: the embeddings for 'dog' and 'puppy' will be close together, while 'dog' and 'car' will be far apart, even though none of these words share letters.

Q18.

What is hallucination in LLMs, and how can it be reduced?

Answer:

Hallucination is when an LLM generates plausible-sounding but factually incorrect or fabricated information. It can be reduced with RAG (grounding answers in real retrieved data), lowering the temperature setting, fine-tuning, and explicit prompt instructions to say 'I don't know' when uncertain.

Why / Explanation:

Mentioning RAG here connects your answer back to the previous question, which signals a coherent understanding rather than memorized flashcards.

Q19.

What is the difference between zero-shot, one-shot, and few-shot prompting?

Answer:

Zero-shot gives the model a task with no examples. One-shot gives exactly one example. Few-shot gives several examples in the prompt to demonstrate the desired pattern before asking the model to perform the task.

Why / Explanation:

This tests practical prompt engineering skill — expect a follow-up like 'when would you use few-shot over zero-shot' (answer: when the task format is unusual or hard to describe in words alone).

Q20.

What is an AI agent, and how is it different from a simple chatbot?

Answer:

A chatbot responds to text with text. An AI agent uses an LLM as a reasoning engine that can decide which tools or actions to call (e.g. search the web, query a database, send an email), observe the results, and decide on next steps in a loop until the task is complete.

Why / Explanation:

The key phrase interviewers listen for is 'tool use' or 'tool calling' plus the idea of a reasoning loop (perceive → reason → act → observe) — this maps directly to how n8n's AI Agent node works.

Section 4 — n8n & Agentic AI Workflows

Q21.

What is n8n, and what makes it different from tools like Zapier?

Answer:

n8n is a visual, node-based workflow automation tool that connects apps and APIs without heavy coding. Unlike Zapier, n8n is open-source, can be self-hosted, and allows more complex logic (branching, loops, custom code nodes) within a single workflow.

Why / Explanation:

Mention self-hosting and flexibility specifically — that's usually the exact distinction interviewers are checking for.

Q22.

What is the AI Agent node in n8n, and how does it work?

Answer:

The AI Agent node lets an LLM act as a decision-maker within a workflow: given a goal, it can choose which connected tools (HTTP Request, Code node, database queries, other workflows) to call, in what order, based on the user's input, rather than following a fixed static path.

Why / Explanation:

This is the single most likely practical question if the role touches n8n AI. Be ready to describe a concrete example, e.g. a support-ticket agent that decides whether to search a knowledge base, escalate to a human, or reply directly.

Q23.

How would you add memory to an n8n AI Agent, and why does it matter?

Answer:

n8n provides Memory nodes (e.g. a simple buffer memory or a database-backed memory) that store previous messages in a conversation and feed them back into the agent's context on each new turn, so the agent can maintain multi-turn context instead of treating every message as brand new.

Why / Explanation:

Without memory, an agent would 'forget' what the user said one message ago — explaining this shows you understand why memory is a distinct architectural piece, not just a nice-to-have.

Q24.

How would you build a RAG pipeline inside n8n?

Answer:

Documents are chunked and converted into embeddings (via an embeddings model node), then stored in a vector store node (e.g. Pinecone or Supabase). At query time, the user's question is embedded and used to retrieve the most relevant chunks, which are passed as context into the AI Agent or Chat Model node to generate a grounded answer.

Why / Explanation:

Walk through it as a pipeline: chunk → embed → store → retrieve → generate. Naming each stage in order is what separates a strong answer from a vague one.

Q25.

How do you reference data between nodes in n8n?

Answer:

n8n passes data between nodes as JSON. You reference a field from a previous node using expression syntax like `{{ $json.fieldName }}`, or reference a specific earlier node with `{{ $('Node Name').item.json.fieldName }}`.

Why / Explanation:

This is a quick practical/syntax check to see if you've actually opened the tool rather than just watched a video about it.

Q26.

How would you handle errors or API failures in an n8n workflow?

Answer:

n8n supports dedicated Error Workflows that trigger automatically when a node fails, so you can log the failure, send an alert, or attempt a retry. Individual nodes also have 'Continue on Fail' and built-in retry settings for transient issues like rate limits.

Why / Explanation:

If you can add a concrete example — e.g. 'retry an LLM API call up to 3 times before falling back to a default response' — it shows practical thinking, not just terminology recall.

Tip: read each answer once, then cover it and try to explain the 'Why' section in your own words out loud — that's what will actually hold up under interview pressure.